
Vocabulary Builder MVP

Phase 2 – Asynchronous Document Processing (Textract)

1. Phase Overview (Business + Technical)

Business Goal

Phase 2 enables the platform to **automatically extract text from uploaded PDFs** in a scalable and cost-efficient way.

This allows the Vocabulary Builder to:

- Accept **real study materials** (PDFs)
- Convert them into **machine-readable text**
- Prepare content for **AI analysis, exercises, and personalization** in later phases

The system is designed to:

- Scale with number of users
 - Avoid blocking or long waits
 - Handle large documents safely
-

2. Why Asynchronous Processing?

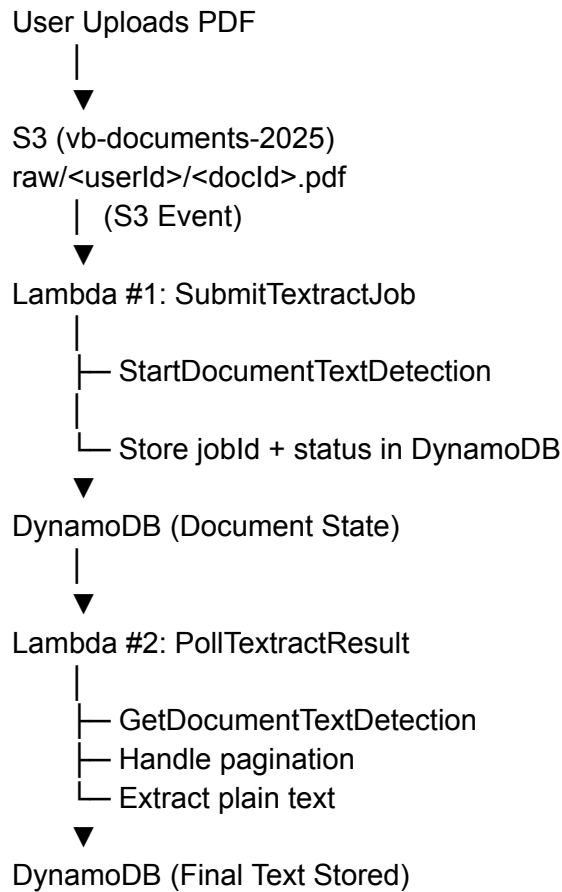
Amazon Textract is an **asynchronous AI service** for PDFs. Instead of waiting for results, the system:

1. Submits a job
2. Continues immediately
3. Fetches results later when ready

Benefits

-  Scales to thousands of documents
 -  Avoids Lambda timeouts
 -  Lower cost
 -  Clear state tracking
-

3. High-Level Architecture (Phase 2)



4. Data Flow (Step-by-Step)

Step 1 – Upload

- User uploads a PDF

File is stored in:

s3://vb-documents-2025/raw/<sub>/<docId>.pdf

-

Step 2 – Job Submission (Lambda #1)

Lambda Name: SubmitTextractJob

Trigger: S3 ObjectCreated event

Responsibility: *Submit Textract job + track metadata*

Actions

- Validates S3 key structure

Calls:

StartDocumentTextDetection

-
- Stores in DynamoDB:
 - textractJobId
 - status = **TEXTRACT_SUBMITTED**
 - s3Key
 - timestamps

Why this Lambda exists

- Keeps upload fast
- Does not wait for Textract
- Clean separation of concerns

Step 3 – Polling & Extraction (Lambda #2)

Lambda Name: PollTextractResult

Trigger: Manual / EventBridge (future Step Function)

Responsibility: *Fetch results + normalize text*

Actions

- Reads jobId from DynamoDB

Calls:

GetDocumentTextDetection

-

- Handles pagination (**NextToken**)
- Extracts text from **LINE** blocks
- Stores result in DynamoDB:
 - `textractText`
 - `status = TEXTRACT_DONE`

Why this Lambda exists

- Textract jobs can take seconds or minutes
- Prevents Lambda timeout
- Allows retries and monitoring

5. DynamoDB State Lifecycle

Each document moves through **clear states**:

Status	Meaning
TEXTRACT_SUBMITTED	Job sent to Textract
TEXTRACT_DONE	Text successfully extracted
ERROR	Failure (retry allowed)

This enables:

- Monitoring
- Retry logic
- Safe orchestration

6. Lambda Responsibility Separation (Key Design Choice)

Lambda	Responsibility
SubmitTextractJob	Submit job only
PollTextractResult	Fetch & process result

- ❌ No Lambda does everything
- ✅ Each Lambda has **one clear purpose**

This design:

- Improves debugging
 - Improves scaling
 - Matches AWS best practices
-

7. Security & Permissions (High Level)

Each Lambda runs with **least-privilege IAM roles**:

Textract Submit Lambda

- textract:StartDocumentTextDetection
- s3:GetObject (raw/)
- dynamodb:UpdateItem

Textract Poll Lambda

- textract:GetDocumentTextDetection
- dynamodb:GetItem / UpdateItem

No public access.

No wildcard permissions.

8. Error Handling & Reliability

- Invalid files are ignored early
 - Textract failures update state
 - DynamoDB condition expressions prevent overwrites
 - Jobs can be retried safely
-

9. Business Value of Phase 2

After Phase 2, the platform can:

- Accept real learning content
- Convert documents into text
- Store structured AI-ready data
- Support future NLP & personalization

This is the **foundation** for:

- Vocabulary extraction
 - Difficulty classification
 - Exercise generation
 - Student-level personalization
-

10. What Comes Next (Phase 3 – Preview)

Phase 3 will introduce:

- Amazon Comprehend (NLP)
- Keyword extraction
- Difficulty scoring
- Exercise generation logic

Phase 2 intentionally stops at clean text extraction.
